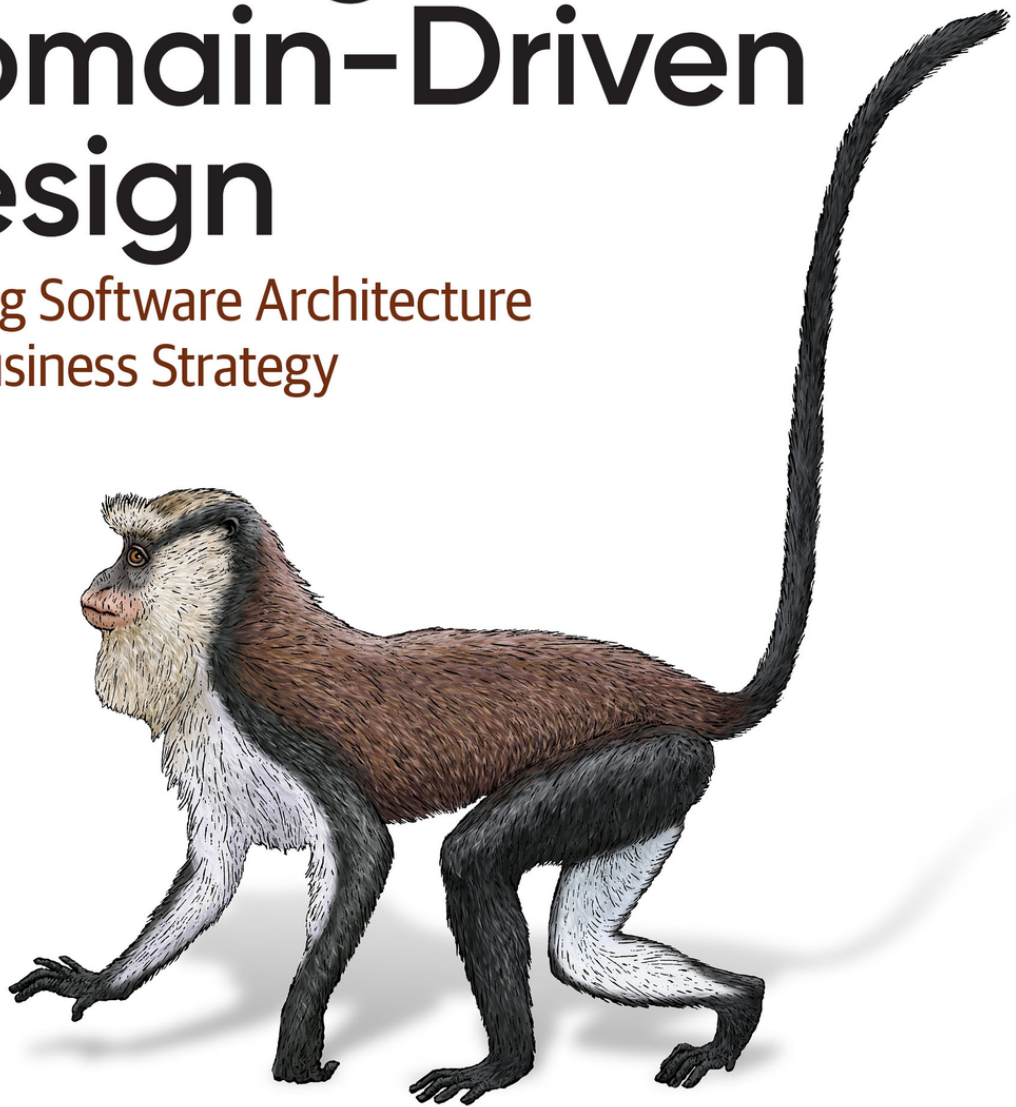


O'REILLY®

Learning Domain-Driven Design

Aligning Software Architecture
and Business Strategy



Vlad Khononov
Foreword by Julie Lerman

Praise for *Learning Domain-Driven Design*

“Vladik Khononov is a unique thinker who has been applying DDD to solve real business problems for years. His ideas constantly move the whole DDD community forward, and this book will inspire beginning DDD practitioners.”

—Nick Tune, Technology Consultant

“Reflecting on my readings of drafts of this book, the thing that comes to mind, with a great deal of joy at the thought, is that it delivers on its title! It is an inviting and informative practice guide, covering the scope of DDD from strategy to technical design. I’ve gained new insight and understanding in areas where I have experience and filled in concepts and practices I’d had less exposure to. Vlad is a wonderful teacher!”

—Ruth Malan, Architecture Consultant at Bredemeyer Consulting

“Vlad has a lot of hard-won experience as a DDD practitioner working on some deeply complex projects and has been generous in sharing that knowledge. In this book, he tells the story of DDD in a unique way providing a great perspective for learning. This book is aimed at newcomers, yet as a longtime DDD practitioner who also writes and speaks about DDD, I found that I learned so much from his perspective.”

—Julie Lerman, Software Coach, O’Reilly Author, and Serial DDD Advocate



Learning Domain-Driven Design

Aligning Software Architecture and Business Strategy

Vlad Khononov

Learning Domain-Driven Design

by Vlad Khononov

Copyright © 2022 Vladislav Khononov. All rights reserved.

Printed in the United States of America.

Published by O’Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

- Acquisitions Editor: Melissa Duffield
- Development Editor: Jill Leonard
- Production Editor: Katherine Tozer
- Copyeditor: Audrey Doyle
- Proofreader: James Fraleigh
- Indexer: Sue Klefstad
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea
- October 2021: First Edition

Revision History for the First Edition

- 2021-10-08: First Release
- 2024-03-22: Second Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098100131> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Learning Domain-Driven Design*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-10013-1

[LSI]

Foreword

Domain-driven design provides a set of practices for a collaborative approach to building software from the perspective of the business—that is, the domain, and its problems that you are targeting. It was originally coined by Eric Evans in 2003 with the publication of what is fondly known in the DDD

community as “The Blue Book.” The book’s title is *Domain-Driven Design: Tackling Complexity in the Heart of Software*.

While tackling complexity and providing a path to clarity is the goal of domain-driven design, there are so many great ideas that can be applied to even less complicated software projects. DDD reminds us that software developers are not the only people involved in building software. The domain experts, for whom the software is being built, bring critical understanding of the problems being solved. We create a partnership throughout the stages of creation as we first apply “strategic design” to understand the business problem, a.k.a. the domain, and break the problem down into smaller, solvable, interconnected problems. The partnership with the domain experts also drives us to communicate in the language of the domain, rather than forcing those on the business side to learn the technical language of software.

The second stage of a DDD-based project is “tactical design,” where we transform the discoveries of strategic design into software architecture and implementation. Again, DDD provides guidance and patterns for organizing these domains and avoiding further complexity. Tactical design continues the partnership with the domain experts who will recognize their domain language even as they look at the code built by the software teams.

Over the years since the publication of “The Blue Book,” not only have many organizations benefited from the ideas, but a community of experienced DDD practitioners has evolved. And the collaborative nature of DDD has resulted in this community sharing their experiences and perspective and creating tools to help teams embrace and benefit from these ideas. In a keynote at Explore DDD in 2019, Eric Evans encouraged the community to continue to evolve DDD—not only its practices but in finding ways to more effectively share its ideas.

And this brings me to why I am such a fan of *Learning Domain-Driven Design*. I was already a fan of Vlad through his conference speaking and other writings. He has a lot of hard-won experience as a DDD practitioner working on some deeply complex projects and has been generous in sharing that knowledge. In this book, he tells the “story” of DDD (not its history, but its concepts) in a unique way, providing a great perspective for learning. This book is aimed at newcomers, yet as a longtime DDD practitioner who also writes and speaks about DDD, I found that I learned so much from his perspective. I was eager to reference his book in my DDD Fundamentals course on Pluralsight before the book was even published and have already been sharing some of this perspective in conversations with clients.

Getting started with DDD can be confusing. Just as we use DDD to reduce the complexity of projects, Vlad presents DDD in a way that reduces the complexity of the topic itself. And he does more than explain the principles of DDD. The latter portion of the book shares some important practices that have evolved from DDD, such as EventStorming, addresses the problem of evolving the business focus or organization and how this might affect the software, and discusses how DDD aligns with microservices and how you can integrate it with a slew of well-known software patterns. I think *Learning Domain-Driven Design* will be an excellent introduction to DDD for newcomers, and a very worthy read for experienced practitioners as well.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Julie Lerman
Software Coach, O'Reilly Author,
and Serial DDD Advocate

Preface

I vividly remember the day I started my first real software engineering job. I was both ecstatic and terrified. After hacking software for local businesses during my high school years, I was eager to become a “real programmer” and write some code for one of the country’s largest outsourcing companies.

In my first days there, my new colleagues were showing me the ropes. After setting up the corporate email and going through the time-tracking system, we finally moved on to the interesting stuff: the company’s coding style and standards. I was told that “here, we always write well-designed code and use the layered architecture.” We went through the definition of each of the three layers—the data access, business logic, and presentation layers—and then discussed the technologies and frameworks for addressing the layers’ needs. Back then, the accepted solution for storing data was Microsoft SQL Server 2000, and it was integrated using ADO.NET in the data access layer. The presentation layer rocked either WinForms for desktop applications or ASP.NET WebForms for the web. We spent quite some time on these two layers, so I was puzzled when the business logic layer didn’t get any attention:

“But what about the business logic layer?”

“That one is straightforward. Here is where you implement the business logic.”

“But what is business logic?”

“Oh, business logic is all the loops and ‘if-else’ statements you need in order to implement the requirements.”

That day I began my journey to find out what exactly business logic is and how on earth it should be implemented in well-designed code. It took me more than three years to finally find the answer.

The answer was in Eric Evans’s seminal book, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. It turned out that I wasn’t wrong. Business logic is indeed important: it is the heart of software! Unfortunately, however, it took me another three years to understand the wisdom Eric shared. The book is very advanced, and the fact that English is my third language didn’t help.

Eventually, though, everything fell into place, and I made peace with the domain-driven design (DDD) methodology. I learned the principles and patterns of DDD, the intricacies of modeling and implementing the business logic, and how to tackle the complexity in the heart of the software that I was building. Despite the obstacles, it definitely was worth it. Getting into domain-driven design was a career-changing experience for me.

Why I Wrote This Book

Over the past 10 years, I have introduced domain-driven design to my colleagues at different companies, conducted in-person classes, and taught online courses. The teaching perspective not only helped me deepen my knowledge, but also allowed me to optimize the way I explain the principles and patterns of domain-driven design.

As often happens, teaching is even more challenging than learning. I'm a huge fan of **Eliyahu M. Goldratt's** work and teachings. Eliyahu used to say that even the most complex systems are inherently simple when viewed from the right angle. During my years of teaching DDD, I was looking for a model of the methodology that would uncover the inherent simplicity of domain-driven design.

This book is the result of my efforts. Its goal is to democratize domain-driven design; make it easier to understand and more accessible to employ. I believe that the DDD methodology is absolutely invaluable, especially when designing modern software systems. This book will give you just enough tools to start applying domain-driven design in your day-to-day work.

Who Should Read This Book

I believe that knowledge of domain-driven design principles and patterns will be useful for software engineers at all levels: junior, senior, staff, and principal. Not only does DDD provide tools and techniques for modeling and effectively implementing software, it also illuminates an often-overlooked aspect of software engineering: the context. Equipped with the knowledge of the system's business problem, you will be much more effective at choosing the appropriate solution. A solution that is not under- or over-engineered, but addresses business needs and goals.

Domain-driven design is even more important for software architects, and even more so for aspiring software architects. Its strategic design decision tools will help you decompose a large system into components—services, microservices, or subsystems—and design how the components are integrated with one another to form a system.

Ultimately, in this book we will discuss not only how to design software, but also how to co-evolve the design with changes in its business context. That crucial aspect of software engineering will help you keep the system's design “in shape” over time and prevent its degradation into a big ball of mud.

Navigating the Book

This book is divided into four parts: strategic design, tactical design, DDD in practice, and DDD's relationships to other methodologies and patterns. In **Part I**, we cover tools and techniques for making large-scale software design decisions. In **Part II**, we focus on the code: the different ways to implement a system's business logic. **Part III** discusses techniques and strategies for applying DDD in real-life

projects. **Part IV** continues the discussion of domain-driven design, but this time in the context of other methodologies and patterns.

Here is a short summary of what you will find in each chapter:

- **Chapter 1** establishes the context of a software engineering project: the business domain, its goals, and how the software is intended to support them.
- **Chapter 2** introduces the notion of a “ubiquitous language”: domain-driven design’s practice for effective communication and knowledge sharing.
- **Chapter 3** discusses how to tackle the complexity of business domains and design the system’s high-level architectural components: bounded contexts.
- **Chapter 4** explores the different patterns of organizing the communication and integration between the bounded contexts.
- **Chapter 5** starts the discussion of business logic implementation patterns with two patterns addressing the cases of simple business logic.
- **Chapter 6** advances from simple to complex business logic and introduces the domain model pattern for tackling its complexity.
- **Chapter 7** adds the perspective of time and introduces an even more advanced way to model and implement business logic: the event-sourced domain model.
- **Chapter 8** shifts the focus to a higher level and describes three architectural patterns for structuring components.
- **Chapter 9** provides the patterns needed to orchestrate the work of the system’s components.
- **Chapter 10** ties together the patterns discussed in the earlier chapters into a number of simple rules of thumb that streamline the process of making design decisions.
- **Chapter 11** explores software design from the perspective of time and how it is supposed to change and evolve through its lifespan.
- **Chapter 12** introduces EventStorming: a low-tech workshop for effectively sharing knowledge, building shared understanding, and designing software.
- **Chapter 13** addresses the difficulties you may face when introducing domain-driven design to brownfield projects.
- **Chapter 14** discusses the relationship between the microservices architectural style and domain-driven design: where they differ and where they complement each other.
- **Chapter 15** explores domain-driven design patterns and tools in the context of the event-driven architecture.

- **Chapter 16** shifts the discussion from operational systems to analytical data management systems and discusses the interplay between domain-driven design and the data mesh architecture.

All of these chapters end with a number of exercise questions to reinforce the learning. Some of the questions use the fictional company “WolfDesk” to demonstrate the various aspects of domain-driven design. Please read the following description of WolfDesk, and return to it when you answer relevant exercise questions.

Example Domain: WolfDesk

WolfDesk provides a help desk tickets management system as a service. If your start-up company needs to provide support to your customers, with WolfDesk’s solution you can get up and running in no time.

WolfDesk uses a different payment model than its competitors. Instead of charging a fee per user, it allows the tenants to set up as many users as needed, and the tenants are charged for the number of support tickets opened per charging period. There is no minimum fee, and there are automatic volume discounts for certain thresholds of monthly tickets: 10% for opening more than 500 tickets, 20% for opening more than 750 tickets, and 30% for opening more than 1,000 tickets per month.

To prevent tenants from abusing the business model, WolfDesk’s ticket lifecycle algorithm ensures that inactive tickets are closed automatically, encouraging customers to open new tickets when further support is needed. Moreover, WolfDesk implements a fraud detection system that analyzes messages and detects cases of unrelated topics being discussed in the same ticket.

To help its tenants streamline the support-related work, WolfDesk has implemented a “support autopilot” feature. The autopilot analyzes new tickets and tries to automatically find a matching solution from the tenant’s ticket history. The functionality allows for further reducing the tickets’ lifespans, encouraging customers to open new tickets for further questions.

WolfDesk incorporates all the security standards and measures to authenticate and authorize its tenants’ users and also allows tenants to configure a single sign-on (SSO) with their existing user management systems.

The administration interface allows tenants to configure the possible values for the tickets’ categories, as well as a list of the tenant’s products that it supports.

To be able to route new tickets to the tenant’s support agents only during their working hours, WolfDesk allows the entry of each agent’s shift schedule.

Since WolfDesk provides its service with no minimal fee, it has to optimize its infrastructure in a way that minimizes the costs of onboarding a new tenant. To do that, WolfDesk leverages serverless computing, which allows it to elastically scale its compute resources based on the operations on active tickets.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

NOTE

This element signifies a general note.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://learning-ddd.com>.

All the code samples presented in the book are implemented in the C# language. Generally, the code samples you see in the chapters are excerpts demonstrating the discussed concepts.

Of course, the concepts and techniques discussed in the book are not limited to the C# language or to the object-oriented programming approach. Everything is relevant for other languages and other programming paradigms. As a result, feel free to implement the book's samples in your favorite language and share them with me. I'll be happy to add them to the book's website.

If you have a technical question or a problem using the code examples, please email bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Learning Domain-Driven Design* by Vlad Khononov (O'Reilly). Copyright 2022 Vladislav Khononov, 978-1-098-10013-1."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://www.oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/lddd>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>

Watch us on YouTube: <https://youtube.com/oreillymedia>

Acknowledgments

Originally, this book was titled “What Is Domain-Driven Design?” and was published as a report in 2019. *Learning Domain-Driven Design* would not have seen the light of day without the report, and I’m obliged to thank those who made “What Is Domain-Driven Design?” possible: Chris Guzikowski, Ryan Shaw, and Alicia Young.¹

This book also wouldn’t have been possible without O’Reilly’s Content Director and Diversity Talent Lead, Melissa Duffield, who championed the project and made it happen. Thank you, Melissa, for all your help!

Jill Leonard was the book’s development editor, project manager, and head coach. Jill’s role in this work cannot be overstated. Jill, thank you so much for all your hard work and help! Extra thanks for keeping me motivated, even when I considered changing my name and hiding in a foreign country.

A huge thanks to the production team for making the book not only writable but readable: Kristen Brown, Audrey Doyle, Kate Dullea, Robert Romano, and Katherine Tozer. For that matter, I want to thank the whole O’Reilly team for the great work you do. It’s a dream come true to be working with you!

Thanks to all the people I interviewed and consulted with: Zsafia Herendi, Scott Hirleman, Trond Hjorteland, Mark Lisker, Chris Richardson, Vaughn Vernon, and Ivan Zakrevsky. Thank you for your wisdom and for being there when I needed help!

Special thanks to the team of reviewers who read through the early drafts and helped me shape the final book: Julie Lerman, Ruth Malan, Diana Montalion, Andrew Padilla, Rodion Promyshlennikov, Viktor Pshenitsyn, Alexei Torunov, Nick Tune, Vasiliy Vasilyuk, and Rebecca Wirfs-Brock. Your support, feedback, and critique helped immensely. Thank you!

I also want to thank Kenny Baas-Schwegler, Alberto Brandolini, Eric Evans, Marco Heimeshoff, Paul Rayner, Mathias Verraes, and the rest of the amazing domain-driven design community. You know who you are. You are my teachers and mentors. Thank you for sharing your knowledge on social media, blogs, and conferences!

I’m most indebted to my dear wife, Vera, for always supporting me in my crazy projects and trying to guard me from things that could distract me from writing. I promise to finally declutter the basement. It is going to happen soon!

Finally, I want to dedicate this book to our beloved Galina Ivanovna Tyumentseva, who supported me so much in this project and whom we sadly lost during the writing of this book. We will always remember you.

#AdoptDontShop

¹ Whenever I mention a group of people, the list is in alphabetical order by last name.

Introduction

Software engineering is hard. To be successful at it, we have to learn continuously, whether it's trying new languages, exploring new technologies, or keeping up with new popular frameworks. However, learning a new JavaScript framework every week is not the hardest aspect of our job. Making sense of new business domains can be far more challenging.

Throughout our careers, it's not uncommon for us to have to develop software for a diverse range of business domains: financial systems, medical software, online retailers, marketing, and many others. In a sense, that is what differentiates our job from most other professions. People working in other fields are often surprised when they find out how much learning is involved in software engineering, especially when changing workplaces.

Failure to grasp the business domain results in suboptimal implementation of the business software. Unfortunately, that's quite common. According to studies, approximately 70% of software projects are not delivered on time, on budget, or according to the client's requirements. In other words, the vast majority of software projects fail. This issue is so deep and widespread that we even have a term for it: software crisis.

The term *software crisis* was introduced all the way back in 1968.¹ One would assume that things would have improved in the intervening 50 years. During those years, numerous approaches, methodologies, and disciplines were introduced to make software engineering more effective: Agile Manifesto, extreme programming, test-driven development, high-level languages, DevOps, and others. Unfortunately, things didn't change much. Projects are still failing quite often and the software crisis is still here.

Many studies have been conducted to investigate the reasons for the common project failures.² Although researchers have not been able to pinpoint a single cause, most of their findings share a common theme: communication. Communication issues thwarting projects can manifest themselves in different ways; for example, unclear requirements, uncertain project goals, or ineffective coordination of effort between teams. Yet again, over the years, we have tried to improve inter- and intrateam communication by introducing new communication opportunities, processes, and mediums. Unfortunately, the success rates of our projects still didn't change much.

Domain-driven design (DDD) proposes to attack the root cause for failed software projects from a different angle. Effective communication is the central theme of the domain-driven design tools and practices you are about to learn in this book. DDD can be divided into two parts: strategic and tactical.

The strategic tools of DDD are used to analyze business domains and strategy, and to foster a shared understanding of the business between the different stakeholders. We will also use this knowledge of the business domain to drive high-level design decisions: decomposing systems into components and defining their integration patterns.

Domain-driven design's tactical tools address a different aspect of communication issues. DDD's tactical patterns allow us to write code in a way that reflects the business domain, addresses its goals, and speaks the language of the business.

Both the strategic and tactical patterns and practices of DDD align software design with its business domain. That's where the name comes from: (business) domain-driven (software) design.

Domain-driven design won't make it possible to install the knowledge of new JavaScript libraries directly into your brain, like in *The Matrix*. However, it will make you a more effective software engineer by alleviating the process of making sense of business domains and guiding the design decisions according to the business strategy. As you will learn in the book's later chapters, the tighter the connection between the software design and its business strategy is, the easier it will be to maintain and evolve the system to meet the future needs of the business, ultimately leading to more successful software projects.

Let's start our DDD journey by exploring the strategic patterns and practices.

¹ “Software Engineering.” Report on a conference sponsored by the NATO Science Committee, Garmisch, Germany, October 7–11, 1968.

² See, for example, Kaur, Rupinder, and Dr. Jyotsna Sengupta (2013), “Software Process Models and Analysis on Failure of Software Development Projects,” <https://arxiv.org/ftp/arxiv/papers/1306/1306.1068.pdf>. See also Sudhakar, Goparaju Purna (2012), “A Model of Critical Success Factors for Software Projects.” *Journal of Enterprise Information Management* 25(6), 537–558.

Part I. Strategic Design

There is no sense in talking about the solution before we agree on the problem, and no sense talking about the implementation steps before we agree on the solution.

—Efrat Goldratt-Ashlag¹

The domain-driven design (DDD) methodology can be divided into two main parts: strategic design and tactical design. The strategic aspect of DDD deals with answering the questions of “what?” and “why?”—what software we are building and why we are building it. The tactical part is all about the “how”—how each component is implemented.

We will begin our journey by exploring domain-driven design patterns and principles of strategic design:

- In **Chapter 1**, you will learn to analyze a company's business strategy: what value it provides to its consumers and how it competes with other companies in the industry. We will identify finer-

grained business building blocks, evaluate their strategic value, and analyze how they affect different software design decisions.

- **Chapter 2** introduces domain-driven design's essential practice for gaining an understanding of the business domain: the *ubiquitous language*. You will learn how to cultivate a ubiquitous language and use it to foster a shared understanding among all project-related stakeholders.
- **Chapter 3** discusses another domain-driven design core tool: the *bounded context* pattern. You will learn why this tool is essential for cultivating a ubiquitous language and how to use it to transform discovered knowledge into a model of the business domain. Ultimately, we will leverage bounded contexts to design coarse-grained components of the software system.
- In **Chapter 4**, you will learn technical and social constraints that affect how system components can be integrated, and integration patterns that address different situations and limitations. We will discuss how each pattern influences collaboration among software development teams and the design of the components' APIs.

The chapter closes by introducing the *context map*: a graphical notation that plots communication between the system's bounded contexts and provides a bird's-eye view of the project's integration and collaboration landscapes.

¹ Goldratt-Ashlag, E. (2010). "The Layers of Resistance—The Buy-In Process According to TOC."

Chapter 1. Analyzing Business Domains

If you are anything like me, you love writing code: solving complex problems, coming up with elegant solutions, and constructing whole new worlds by carefully crafting their rules, structures, and behavior. I believe that's what interested you in domain-driven design (DDD): you want to be better at your craft. This chapter, however, has nothing to do with writing code. In this chapter, you will learn how companies work: why they exist, what goals they are pursuing, and their strategies for achieving their goals.

When I teach this material in my domain-driven design classes, many students actually ask, "Do we need to know this material? We are writing software, not running businesses." The answer to their question is a resounding "yes." To design and build an effective solution, you have to understand the problem. The problem, in our context, is the software system we have to build. To understand the problem, you have to understand the context within which it exists—the organization's business strategy, and what value it seeks to gain by building the software.

In this chapter, you will learn domain-driven design tools for analyzing a company's business domain and its structure: its core, supporting, and generic subdomains. This material is the groundwork for

designing software. In the remaining chapters, you will learn the different ways these concepts affect software design.

What Is a Business Domain?

A business domain defines a company's main area of activity. Generally speaking, it's the service the company provides to its clients. For example:

- FedEx provides courier delivery.
- Starbucks is best known for its coffee.
- Walmart is one of the most widely recognized retail establishments.

A company can operate in multiple business domains. For example, Amazon provides both retail and cloud computing services. Uber is a rideshare company that also provides food delivery and bicycle-sharing services.

It's important to note that companies may change their business domains often. A canonical example of this is Nokia, which over the years has operated in fields as diverse as wood processing, rubber manufacturing, telecommunications, and mobile communications.

What Is a Subdomain?

To achieve its business domain's goals and targets, a company has to operate in multiple *subdomains*. A subdomain is a fine-grained area of business activity. All of a company's subdomains form its business domain: the service it provides to its customers. Implementing a single subdomain is not enough for a company to succeed; it's just one building block in the overarching system. The subdomains have to interact with each other to achieve the company's goals in its business domain. For example, Starbucks may be most recognized for its coffee, but building a successful coffeehouse chain requires more than just knowing how to make great coffee. You also have to buy or rent real estate at effective locations, hire personnel, and manage finances, among other activities. None of these subdomains on its own will make a profitable company. All of them together are necessary for a company to be able to compete in its business domain(s).

Types of Subdomains

Just as a software system comprises various architectural components—databases, frontend applications, backend services, and others—subdomains bear different strategic/business values. Domain-driven design distinguishes between three types of subdomains: core, generic, and supporting. Let's see how they differ from a company strategy point of view.

Core subdomains

A *core subdomain* is what a company does differently from its competitors. This may involve inventing new products or services or reducing costs by optimizing existing processes.

Let's take Uber as an example. Initially, the company provided a novel form of transportation: ridesharing. As its competitors caught up, Uber found ways to optimize and evolve its core business: for example, reducing costs by matching riders heading in the same direction.

Uber's core subdomains affect its bottom line. This is how the company differentiates itself from its competitors. This is the company's strategy for providing better service to its customers and/or maximizing its profitability. To maintain a competitive advantage, core subdomains involve inventions, smart optimizations, business know-how, or other intellectual property.

Consider another example: Google Search's ranking algorithm. At the time of this writing, Google's advertising platform accounts for the majority of its profits. That said, Google Ads is not a subdomain, but rather a separate business domain with subdomains comprising it, among its cloud computing service (Google Cloud Platform), productivity and collaboration tools (Google Workspaces), and other fields in which Alphabet, Google's parent company, operates. But what about Google Search and its ranking algorithm? Although the search engine is not a paid service, it serves as the largest display platform for Google Ads. Its ability to provide excellent search results is what drives traffic, and subsequently, it is an important component of the Ads platform. Serving suboptimal search results due to a bug in the algorithm or a competitor coming up with an even better search service will hurt the ad business's revenue. So, for Google, the ranking algorithm is a core subdomain.

Complexity

A core subdomain that is simple to implement can only provide a short-lived competitive advantage. Therefore, core subdomains are naturally complex. Continuing with the Uber example, the company not only created a new market space with ridesharing, it disrupted a decades-old monolithic architecture, the taxi industry, through targeted use of technology. By understanding its business domain, Uber was able to design a more reliable and transparent method of transportation. There should be high entry barriers for a company's core business; it should be hard for competitors to copy or imitate the company's solution.

Sources of competitive advantage

It's important to note that core subdomains are not necessarily technical. Not all business problems are solved through algorithms or other technical solutions. A company's competitive advantage can come from various sources.

Consider, for example, a jewelry maker selling its products online. The online shop is important, but it's not a core subdomain. The jewelry design is. The company can use an existing off-the-shelf online shop engine, but it cannot outsource the design of its jewelry. The design is the reason customers buy the jewelry maker's products and remember the brand.

As a more intricate example, imagine a company that specializes in *manual* fraud detection. The company trains its analysts to go over questionable documents and flag potential fraud cases. You are building the software system the analysts are working with. Is it a core subdomain? No. The core subdomain is the work the analysts are doing. The system you are building has nothing to do with fraud analysis, it just displays the documents and tracks the analysts' comments.

CORE SUBDOMAIN VERSUS CORE DOMAIN

Core subdomains are also called core domains. For example, in the original domain-driven design book, Eric Evans uses “core subdomain” and “core domain” interchangeably. Although the term “core domain” is used often, I prefer to use “core subdomain” for a number of reasons. First, it is a *subdomain*, and I prefer to avoid confusion with *business domains*. Second, as you will learn in **Chapter 11**, it's not uncommon for subdomains to evolve over time and change their types. For example, a core subdomain can turn into a generic subdomain. Hence, saying that “a *generic* subdomain has evolved into a *core* subdomain” is more straightforward than saying “a *generic subdomain* has evolved into a *core domain*.”

Generic subdomains

Generic subdomains are business activities that all companies are performing in the same way. Like core subdomains, generic subdomains are generally complex and hard to implement. However, generic subdomains do not provide any competitive edge for the company. There is no need for innovation or optimization here: battle-tested implementations are widely available, and all companies use them.

For example, most systems need to authenticate and authorize their users. Instead of inventing a proprietary authentication mechanism, it makes more sense to use an existing solution. Such a solution is likely to be more reliable and secure since it has already been tested by many other companies that have the same needs.

Going back to the example of a jewelry maker selling its products online, jewelry design is a core subdomain, but the online shop is a generic subdomain. Using the same online retail platform—the same generic solution—as its competitors would not impact the jewelry maker's competitive advantage.

Supporting subdomains

As the name suggests, *supporting subdomains* support the company's business. However, contrary to core subdomains, supporting subdomains do not provide any competitive advantage.

For example, consider an online advertising company whose core subdomains include matching ads to visitors, optimizing the ads' effectiveness, and minimizing the cost of ad space. However, to achieve success in these areas, the company needs to catalog its creative materials. The way the company stores and indexes its physical creative materials, such as banners and landing pages, does not impact its profits. There is nothing to invent or optimize in that area. On the other hand, the creative catalog is

essential for implementing the company's advertising management and serving systems. That makes the content cataloging solution one of the company's supporting subdomains.

This document was truncated here because it was created in the Evaluation Mode.

